



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

Describing Meaning LI7869

Dr. Thomas Stephen

School of Linguistic, Speech and Communication Sciences
Trinity College Dublin

Semester 2 2026

Roadmap

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - ① Individuals and denotation brackets $\llbracket \cdot \rrbracket$

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - 1 Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - 2 Predicates as **sets**; predication as **membership**

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - ① Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - ② Predicates as **sets**; predication as **membership**
 - ③ Why set theory is the right meta-language

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - ① Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - ② Predicates as **sets**; predication as **membership**
 - ③ Why set theory is the right meta-language
 - ④ Set theory basics: sets, membership, subsets, operations

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - 1 Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - 2 Predicates as **sets**; predication as **membership**
 - 3 Why set theory is the right meta-language
 - 4 Set theory basics: sets, membership, subsets, operations
 - 5 Intersective modification as **intersection**

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - ① Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - ② Predicates as **sets**; predication as **membership**
 - ③ Why set theory is the right meta-language
 - ④ Set theory basics: sets, membership, subsets, operations
 - ⑤ Intersective modification as **intersection**
 - ⑥ Transitives as **relations**: ordered pairs, Cartesian products

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - ① Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - ② Predicates as **sets**; predication as **membership**
 - ③ Why set theory is the right meta-language
 - ④ Set theory basics: sets, membership, subsets, operations
 - ⑤ Intersective modification as **intersection**
 - ⑥ Transitives as **relations**: ordered pairs, Cartesian products
 - ⑦ Functions as a special kind of relation

Today

- **Goal:** extend last week's truth-functional picture to **subject–predicate structure**.
- **Plan:**
 - ① Individuals and denotation brackets $\llbracket \cdot \rrbracket$
 - ② Predicates as **sets**; predication as **membership**
 - ③ Why set theory is the right meta-language
 - ④ Set theory basics: sets, membership, subsets, operations
 - ⑤ Intersective modification as **intersection**
 - ⑥ Transitives as **relations**: ordered pairs, Cartesian products
 - ⑦ Functions as a special kind of relation

Framing: phenomenon, tools, comparison

- **Phenomenon:** sentences fundamentally decompose into **subjects** and **predicates** corresponding to **individuals** and **sets**.

Framing: phenomenon, tools, comparison

- **Phenomenon:** sentences fundamentally decompose into **subjects** and **predicates** corresponding to **individuals** and **sets**.
- **Formal tools:** set theory (membership, subethood, intersection/union, relations, functions).

Framing: phenomenon, tools, comparison

- **Phenomenon:** sentences fundamentally decompose into **subjects** and **predicates** corresponding to **individuals** and **sets**.
- **Formal tools:** set theory (membership, subsethood, intersection/union, relations, functions).
- **Cross-linguistic comparison:** predication is realised differently (including cases with no overt copula), but the same semantic backbone applies.

Framing: phenomenon, tools, comparison

- **Phenomenon:** sentences fundamentally decompose into **subjects** and **predicates** corresponding to **individuals** and **sets**.
- **Formal tools:** set theory (membership, subsethood, intersection/union, relations, functions).
- **Cross-linguistic comparison:** predication is realised differently (including cases with no overt copula), but the same semantic backbone applies.

Motivation: beyond truth functions

Motivation: beyond truth functions

Quick recap: what propositional logic bought us

- In propositional logic, we treated whole sentences as **atomic propositions** $(p, q, \dots)$.

Quick recap: what propositional logic bought us

- In propositional logic, we treated whole sentences as **atomic propositions** $(p, q, \dots)$.
- Connectives like *and*, *not*, *or* were modelled as **truth functions**.

Quick recap: what propositional logic bought us

- In propositional logic, we treated whole sentences as **atomic propositions** $(p, q, \dots)$.
- Connectives like *and*, *not*, *or* were modelled as **truth functions**.
- So: the truth of a complex sentence is a **function of the truth** of its parts.

Quick recap: what propositional logic bought us

- In propositional logic, we treated whole sentences as **atomic propositions** $(p, q, \dots)$.
- Connectives like *and*, *not*, *or* were modelled as **truth functions**.
- So: the truth of a complex sentence is a **function of the truth** of its parts.

But: we also want to model sub-sentential meaning

- Consider:

But: we also want to model sub-sentential meaning

- Consider:
 - *Neil giggles.*

But: we also want to model sub-sentential meaning

- Consider:
 - *Neil giggles.*
 - *Anna is smart.*

But: we also want to model sub-sentential meaning

- Consider:
 - *Neil giggles.*
 - *Anna is smart.*
 - *Fiona is a dog.*

But: we also want to model sub-sentential meaning

- Consider:
 - *Neil giggles.*
 - *Anna is smart.*
 - *Fiona is a dog.*
- The parts *giggles*, *smart*, *dog* are not (by themselves) true or false.

But: we also want to model sub-sentential meaning

- Consider:
 - *Neil giggles.*
 - *Anna is smart.*
 - *Fiona is a dog.*
- The parts *giggles*, *smart*, *dog* are not (by themselves) true or false.
- This motivates a new semantic **object**: a **predicate**.

But: we also want to model sub-sentential meaning

- Consider:
 - *Neil giggles.*
 - *Anna is smart.*
 - *Fiona is a dog.*
- The parts *giggles*, *smart*, *dog* are not (by themselves) true or false.
- This motivates a new semantic **object**: a **predicate**.

Individuals and denotation brackets

Denotation brackets

- We use denotation brackets $\llbracket \cdot \rrbracket$ to talk about what an expression **denotes**.

Denotation brackets

- We use denotation brackets $\llbracket \cdot \rrbracket$ to talk about what an expression **denotes**.
- Roughly: $\llbracket \alpha \rrbracket$ is “the semantic value” of the expression α .

Denotation brackets

- We use denotation brackets $\llbracket \cdot \rrbracket$ to talk about what an expression **denotes**.
- Roughly: $\llbracket \alpha \rrbracket$ is “the semantic value” of the expression α .
- Example: proper names are standardly treated as denoting **individuals**.

Denotation brackets

- We use denotation brackets $\llbracket \cdot \rrbracket$ to talk about what an expression **denotes**.
- Roughly: $\llbracket \alpha \rrbracket$ is “the semantic value” of the expression α .
- Example: proper names are standardly treated as denoting **individuals**.

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .
- We write:

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .
- We write:

$$\llbracket \text{Neil} \rrbracket = n$$

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .
- We write:

$$\llbracket \text{Neil} \rrbracket = n$$

- Similarly:

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .
- We write:

$$\llbracket \textit{Neil} \rrbracket = n$$

- Similarly:

$$\llbracket \textit{Anna} \rrbracket = a \qquad \llbracket \textit{Fiona} \rrbracket = f$$

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .
- We write:

$$\llbracket \textit{Neil} \rrbracket = n$$

- Similarly:

$$\llbracket \textit{Anna} \rrbracket = a \qquad \llbracket \textit{Fiona} \rrbracket = f$$

- The main point: names pick out particular individuals.

Proper names denote individuals

- Let the proper name *Neil* correspond to an individual constant n .
- We write:

$$\llbracket \textit{Neil} \rrbracket = n$$

- Similarly:

$$\llbracket \textit{Anna} \rrbracket = a \qquad \llbracket \textit{Fiona} \rrbracket = f$$

- The main point: names pick out particular individuals.

Individuals in the metalanguage

- In the formal metalanguage we use lower-case letters ($a, b, c, n, f, \dots$) as individual constants.

Individuals in the metalanguage

- In the formal metalanguage we use lower-case letters ($a, b, c, n, f, \dots$) as individual constants.
- These are not English words; they are just names for individuals in our model.

Individuals in the metalanguage

- In the formal metalanguage we use lower-case letters ($a, b, c, n, f, \dots$) as individual constants.
- These are not English words; they are just names for individuals in our model.
- We will use these when we translate English into a more explicit semantic notation.

Individuals in the metalanguage

- In the formal metalanguage we use lower-case letters ($a, b, c, n, f, \dots$) as individual constants.
- These are not English words; they are just names for individuals in our model.
- We will use these when we translate English into a more explicit semantic notation.

Predicates as sets; predication as membership

Predicates as sets; predication as membership

Predicates as sets

- A one-place predicate corresponds to a **set** of individuals.

Predicates as sets

- A one-place predicate corresponds to a **set** of individuals.
- Example: *giggles* picks out the gigglers:

Predicates as sets

- A one-place predicate corresponds to a **set** of individuals.
- Example: *giggles* picks out the gigglers:

$$\llbracket \textit{giggles} \rrbracket = \{x : x \text{ giggles}\}$$

Predicates as sets

- A one-place predicate corresponds to a **set** of individuals.
- Example: *giggles* picks out the gigglers:

$$\llbracket \textit{giggles} \rrbracket = \{x : x \text{ giggles}\}$$

- Predication becomes **set membership**.

Predicates as sets

- A one-place predicate corresponds to a **set** of individuals.
- Example: *giggles* picks out the gigglers:

$$\llbracket \textit{giggles} \rrbracket = \{x : x \text{ giggles}\}$$

- Predication becomes **set membership**.

Two equivalent notations for predication

- Predicate notation (first-order style):

Two equivalent notations for predication

- Predicate notation (first-order style):

giggles(*n*)

Two equivalent notations for predication

- Predicate notation (first-order style):

giggles(*n*)

- Set-membership notation:

Two equivalent notations for predication

- Predicate notation (first-order style):

giggles(*n*)

- Set-membership notation:

$n \in \llbracket \textit{giggles} \rrbracket$ where $\llbracket \textit{giggles} \rrbracket = \{x : x \textit{ giggles}\}$

Two equivalent notations for predication

- Predicate notation (first-order style):

giggles(*n*)

- Set-membership notation:

$n \in \llbracket \textit{giggles} \rrbracket$ where $\llbracket \textit{giggles} \rrbracket = \{x : x \textit{ giggles}\}$

- These are equivalent ways of packaging the same truth conditions.

Two equivalent notations for predication

- Predicate notation (first-order style):

giggles(*n*)

- Set-membership notation:

$n \in \llbracket \textit{giggles} \rrbracket$ where $\llbracket \textit{giggles} \rrbracket = \{x : x \textit{ giggles}\}$

- These are equivalent ways of packaging the same truth conditions.

Concrete translations: subject–predicate sentences

Concrete translations: subject–predicate sentences

Intransitives: *Neil giggles*

- Sentence: *Neil giggles*.

Intransitives: *Neil giggles*

- Sentence: *Neil giggles*.
- Let $\llbracket \text{Neil} \rrbracket = n$.

Intransitives: *Neil giggles*

- Sentence: *Neil giggles*.
- Let $\llbracket \text{Neil} \rrbracket = n$.
- Predicate notation:

Intransitives: *Neil giggles*

- Sentence: *Neil giggles*.
- Let $\llbracket \text{Neil} \rrbracket = n$.
- Predicate notation:

giggles(n)

Intransitives: *Neil giggles*

- Sentence: *Neil giggles*.
- Let $\llbracket \text{Neil} \rrbracket = n$.
- Predicate notation:

giggles(n)

- Set translation:

Intransitives: *Neil giggles*

- Sentence: *Neil giggles*.
- Let $\llbracket \text{Neil} \rrbracket = n$.
- Predicate notation:

giggles(n)

- Set translation:

$n \in \{x : x \text{ giggles}\}$

Predicate adjectives: *Anna is smart*

- Sentence: *Anna is smart.*

Predicate adjectives: *Anna is smart*

- Sentence: *Anna is smart*.
- Let $\llbracket \textit{Anna} \rrbracket = a$.

Predicate adjectives: *Anna is smart*

- Sentence: *Anna is smart*.
- Let $\llbracket Anna \rrbracket = a$.
- Predicate notation:

Predicate adjectives: *Anna is smart*

- Sentence: *Anna is smart*.
- Let $\llbracket Anna \rrbracket = a$.
- Predicate notation:

smart(*a*)

Predicate adjectives: *Anna is smart*

- Sentence: *Anna is smart*.
- Let $\llbracket Anna \rrbracket = a$.
- Predicate notation:

smart(*a*)

- Set translation:

Predicate adjectives: *Anna is smart*

- Sentence: *Anna is smart*.
- Let $\llbracket Anna \rrbracket = a$.
- Predicate notation:

smart(*a*)

- Set translation:

$a \in \{x : x \text{ is smart}\}$

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.
- Predicate notation:

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.
- Predicate notation:

$\textit{dog}(f)$

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.
- Predicate notation:

$\textit{dog}(f)$

- Set translation:

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.
- Predicate notation:

$$\textit{dog}(f)$$

- Set translation:

$$f \in \{x : x \text{ is a dog}\}$$

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.
- Predicate notation:

$$\textit{dog}(f)$$

- Set translation:

$$f \in \{x : x \text{ is a dog}\}$$

- (We ignore *a* for now; determiners come later.)

Predicate nominals: *Fiona is a dog*

- Sentence: *Fiona is a dog*.
- Let $\llbracket \textit{Fiona} \rrbracket = f$.
- Predicate notation:

$$\textit{dog}(f)$$

- Set translation:

$$f \in \{x : x \text{ is a dog}\}$$

- (We ignore *a* for now; determiners come later.)

Exercise (warm-up)

Translate each sentence into set-theoretic form, using membership ($\in$). Use constants m, r, k for the individuals.

- 1 *Mina sleeps.*

Exercise (warm-up)

Translate each sentence into set-theoretic form, using membership ($\in$). Use constants m, r, k for the individuals.

- ① *Mina sleeps.*
- ② *Rory is tall.*

Exercise (warm-up)

Translate each sentence into set-theoretic form, using membership ($\in$). Use constants m, r, k for the individuals.

- ① *Mina sleeps.*
- ② *Rory is tall.*
- ③ *Kiki is a cat.*

Exercise (warm-up)

Translate each sentence into set-theoretic form, using membership ($\in$). Use constants m, r, k for the individuals.

- 1 *Mina sleeps.*
- 2 *Rory is tall.*
- 3 *Kiki is a cat.*

Intersective modification

Intersective modification: *a happy dog*

- Many adjective–noun combinations behave **intersectively**.

Intersective modification: *a happy dog*

- Many adjective–noun combinations behave **intersectively**.
- Set translation (first-pass):

Intersective modification: *a happy dog*

- Many adjective–noun combinations behave **intersectively**.
- Set translation (first-pass):

$$\llbracket \text{happy dog} \rrbracket = \llbracket \text{happy} \rrbracket \cap \llbracket \text{dog} \rrbracket$$

Intersective modification: *a happy dog*

- Many adjective–noun combinations behave **intersectively**.
- Set translation (first-pass):

$$\llbracket \textit{happy dog} \rrbracket = \llbracket \textit{happy} \rrbracket \cap \llbracket \textit{dog} \rrbracket$$

- So:

$$\textit{Fiona is a happy dog is true} \iff f \in \llbracket \textit{happy} \rrbracket \cap \llbracket \textit{dog} \rrbracket$$

Intersective modification: *a happy dog*

- Many adjective–noun combinations behave **intersectively**.
- Set translation (first-pass):

$$\llbracket \text{happy dog} \rrbracket = \llbracket \text{happy} \rrbracket \cap \llbracket \text{dog} \rrbracket$$

- So:

$$\text{Fiona is a happy dog is true} \iff f \in \llbracket \text{happy} \rrbracket \cap \llbracket \text{dog} \rrbracket$$

- Equivalently:

$$f \in \llbracket \text{happy} \rrbracket \text{ and } f \in \llbracket \text{dog} \rrbracket$$

Exercise (intersection)

Translate into set notation using $\cap$. Use constants m, k for the individuals.

- 1 *Mina is a tall student.*

Exercise (intersection)

Translate into set notation using $\cap$. Use constants m, k for the individuals.

- ① *Mina is a tall student.*
- ② *Kiki is a sleepy cat.*

Exercise (intersection)

Translate into set notation using $\cap$. Use constants m, k for the individuals.

- ① *Mina is a tall student.*
- ② *Kiki is a sleepy cat.*

Cross-linguistic predication

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).
- Examples (Latin-alphabet transliteration):

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).
- Examples (Latin-alphabet transliteration):
 - **Arabic (MSA):** *al-walad-u Tawiil-un* 'the boy (is) tall'

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).
- Examples (Latin-alphabet transliteration):
 - **Arabic (MSA):** *al-walad-u Tawiil-un* 'the boy (is) tall'
 - **Arabic (MSA):** *Faatima Taaliba* 'Fatima (is) a student'

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).
- Examples (Latin-alphabet transliteration):
 - **Arabic (MSA):** *al-walad-u Tawiil-un* ‘the boy (is) tall’
 - **Arabic (MSA):** *Faatima Taaliba* ‘Fatima (is) a student’
 - **Mandarin:** *Tā hěn gāo* ‘He (is) tall’

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).
- Examples (Latin-alphabet transliteration):
 - **Arabic (MSA):** *al-walad-u Tawiil-un* ‘the boy (is) tall’
 - **Arabic (MSA):** *Faatima Taaliba* ‘Fatima (is) a student’
 - **Mandarin:** *Tā hěn gāo* ‘He (is) tall’
 - **Mandarin:** *Tā shì lǎoshī* ‘He is a teacher’

Cross-linguistic: no overt copula (examples)

- Some languages allow **predication without an overt copula** (especially in the present tense).
- Examples (Latin-alphabet transliteration):
 - Arabic (MSA):** *al-walad-u Tawiil-un* 'the boy (is) tall'
 - Arabic (MSA):** *Faatima Taaliba* 'Fatima (is) a student'
 - Mandarin:** *Tā hěn gāo* 'He (is) tall'
 - Mandarin:** *Tā shì lǎoshī* 'He is a teacher'

English also allows copula omission: small clauses

- Even in English, the copula can be **omissible** in certain environments (often analysed as **small clauses**).

English also allows copula omission: small clauses

- Even in English, the copula can be **omissible** in certain environments (often analysed as **small clauses**).
- With *consider*:

English also allows copula omission: small clauses

- Even in English, the copula can be **omissible** in certain environments (often analysed as **small clauses**).
- With *consider*:
 - *I consider Anna smart.*

English also allows copula omission: small clauses

- Even in English, the copula can be **omissible** in certain environments (often analysed as **small clauses**).
- With *consider*:
 - *I consider Anna smart.*
 - *I consider Fiona a dog.*

English also allows copula omission: small clauses

- Even in English, the copula can be **omissible** in certain environments (often analysed as **small clauses**).
- With *consider*:
 - *I consider Anna smart.*
 - *I consider Fiona a dog.*
 - *I consider Neil a genius.*

English also allows copula omission: small clauses

- Even in English, the copula can be **omissible** in certain environments (often analysed as **small clauses**).
- With *consider*:
 - *I consider Anna smart.*
 - *I consider Fiona a dog.*
 - *I consider Neil a genius.*

Set theory: the meta-language

Set theory: the meta-language

Why set theory?

- Truth tables gave us a precise meta-language for **connectives**.

Why set theory?

- Truth tables gave us a precise meta-language for **connectives**.
- Set theory gives us a precise meta-language for **predicates**:

Why set theory?

- Truth tables gave us a precise meta-language for **connectives**.
- Set theory gives us a precise meta-language for **predicates**:
 - predication as membership ($\in$)

Why set theory?

- Truth tables gave us a precise meta-language for **connectives**.
- Set theory gives us a precise meta-language for **predicates**:
 - predication as membership ($\in$)
 - interjective modification as intersection ($\cap$)

Why set theory?

- Truth tables gave us a precise meta-language for **connectives**.
- Set theory gives us a precise meta-language for **predicates**:
 - predication as membership ($\in$)
 - intersective modification as intersection ($\cap$)
 - (soon) determiners as relations between sets (subthood, non-empty intersection, etc.)

Why set theory?

- Truth tables gave us a precise meta-language for **connectives**.
- Set theory gives us a precise meta-language for **predicates**:
 - predication as membership ($\in$)
 - intersective modification as intersection ($\cap$)
 - (soon) determiners as relations between sets (subthood, non-empty intersection, etc.)

Defining sets

- Sets are collections of objects called their **members** (or **elements**).

Defining sets

- Sets are collections of objects called their **members** (or **elements**).
- Sets are defined only by their members: if two sets have exactly the same members, they are the same set.

Defining sets

- Sets are collections of objects called their **members** (or **elements**).
- Sets are defined only by their members: if two sets have exactly the same members, they are the same set.
- We write sets using curly brackets: $\{ \}$

Defining sets

- Sets are collections of objects called their **members** (or **elements**).
- Sets are defined only by their members: if two sets have exactly the same members, they are the same set.
- We write sets using curly brackets: $\{ \}$
- $\{1\}$ is read as “the set containing 1”.

Defining sets

- Sets are collections of objects called their **members** (or **elements**).
- Sets are defined only by their members: if two sets have exactly the same members, they are the same set.
- We write sets using curly brackets: $\{ \}$
- $\{1\}$ is read as “the set containing 1”.
- Sets can contain all kinds of things (including other sets).

Defining sets

- Sets are collections of objects called their **members** (or **elements**).
- Sets are defined only by their members: if two sets have exactly the same members, they are the same set.
- We write sets using curly brackets: $\{ \}$
- $\{1\}$ is read as “the set containing 1”.
- Sets can contain all kinds of things (including other sets).

Extensional vs. intensional definitions

- **Extensional** (list the members):

Extensional vs. intensional definitions

- **Extensional** (list the members):

$\{2, 3, 5, 7\}$

Extensional vs. intensional definitions

- **Extensional** (list the members):

$\{2, 3, 5, 7\}$

- **Intensional** / set-builder (describe the members):

Extensional vs. intensional definitions

- **Extensional** (list the members):

$\{2, 3, 5, 7\}$

- **Intensional** / set-builder (describe the members):

$\{x : x \text{ is a prime number} \leq 10\}$

Extensional vs. intensional definitions

- **Extensional** (list the members):

$\{2, 3, 5, 7\}$

- **Intensional** / set-builder (describe the members):

$\{x : x \text{ is a prime number} \leq 10\}$

- Once defined, we often give a set a name:

Extensional vs. intensional definitions

- **Extensional** (list the members):

$$\{2, 3, 5, 7\}$$

- **Intensional** / set-builder (describe the members):

$$\{x : x \text{ is a prime number} \leq 10\}$$

- Once defined, we often give a set a name:

$$A = \{1, b\}$$

The empty set

- The **empty set** contains no members.

The empty set

- The **empty set** contains no members.
- Notation: $\{\}$ or $\emptyset$.

The empty set

- The **empty set** contains no members.
- Notation: $\{\}$ or $\emptyset$.

Order and repetition don't matter (for sets)

- Sets are **not ordered**: $\{2, 5, 7, 4\} = \{5, 2, 4, 7\}$.

Order and repetition don't matter (for sets)

- Sets are **not ordered**: $\{2, 5, 7, 4\} = \{5, 2, 4, 7\}$.
- Repetition does not change a set: $\{3, 3, 3\} = \{3\}$.

Order and repetition don't matter (for sets)

- Sets are **not ordered**: $\{2, 5, 7, 4\} = \{5, 2, 4, 7\}$.
- Repetition does not change a set: $\{3, 3, 3\} = \{3\}$.
- A set with one member is a **singleton**: $\{3\}$.

Order and repetition don't matter (for sets)

- Sets are **not ordered**: $\{2, 5, 7, 4\} = \{5, 2, 4, 7\}$.
- Repetition does not change a set: $\{3, 3, 3\} = \{3\}$.
- A set with one member is a **singleton**: $\{3\}$.

Set relations

Membership $\in$

- $x \in A$ means: x is a member of the set A .

Membership $\in$

- $x \in A$ means: x is a member of the set A .
- Example: if $A = \{1, 2, 3\}$, then:

Membership $\in$

- $x \in A$ means: x is a member of the set A .
- Example: if $A = \{1, 2, 3\}$, then:

$2 \in A$ is true $5 \in A$ is false

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .
- Example: let $A = \{1, 2\}$, $B = \{2, n\}$, $C = \{1, 2, m\}$.

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .
- Example: let $A = \{1, 2\}$, $B = \{2, n\}$, $C = \{1, 2, m\}$.
 - $A \subseteq C$ is true

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .
- Example: let $A = \{1, 2\}$, $B = \{2, n\}$, $C = \{1, 2, m\}$.
 - $A \subseteq C$ is true
 - $A \subseteq B$ is false

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .
- Example: let $A = \{1, 2\}$, $B = \{2, n\}$, $C = \{1, 2, m\}$.
 - $A \subseteq C$ is true
 - $A \subseteq B$ is false
 - $B \subseteq C$ is false

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .
- Example: let $A = \{1, 2\}$, $B = \{2, n\}$, $C = \{1, 2, m\}$.
 - $A \subseteq C$ is true
 - $A \subseteq B$ is false
 - $B \subseteq C$ is false
- Every set is a subset of itself.

Subsethood $\subseteq$

- $A \subseteq B$ means: every member of A is also a member of B .
- Example: let $A = \{1, 2\}$, $B = \{2, n\}$, $C = \{1, 2, m\}$.
 - $A \subseteq C$ is true
 - $A \subseteq B$ is false
 - $B \subseteq C$ is false
- Every set is a subset of itself.

Proper subset $\subset$

- $A \subset B$ means: $A \subseteq B$ and $A \neq B$.

Proper subset $\subset$

- $A \subset B$ means: $A \subseteq B$ and $A \neq B$.

Set operations

Union $\cup$

- $A \cup B$ is the set containing everything that is in A **or** in B (inclusive OR).

Union $\cup$

- $A \cup B$ is the set containing everything that is in A **or** in B (inclusive OR).
- Example: if $A = \{1, 2\}$ and $B = \{2, 3, b\}$, then:

Union $\cup$

- $A \cup B$ is the set containing everything that is in A **or** in B (inclusive OR).
- Example: if $A = \{1, 2\}$ and $B = \{2, 3, b\}$, then:

$$A \cup B = \{1, 2, 3, b\}$$

Intersection $\cap$

- $A \cap B$ is the set containing everything that is in A **and** in B .

Intersection $\cap$

- $A \cap B$ is the set containing everything that is in A **and** in B .
- Example: if $A = \{1, 2\}$ and $B = \{2, 3, b\}$, then:

Intersection $\cap$

- $A \cap B$ is the set containing everything that is in A **and** in B .
- Example: if $A = \{1, 2\}$ and $B = \{2, 3, b\}$, then:

$$A \cap B = \{2\}$$

Complement / difference \

- $A \setminus B$ is the set of things in A that are **not** also in B .

Complement / difference \

- $A \setminus B$ is the set of things in A that are **not** also in B .
- Example: if $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$, then:

Complement / difference \

- $A \setminus B$ is the set of things in A that are **not** also in B .
- Example: if $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$, then:

$$A \setminus B = \{1, 4\}$$

Cardinality | · |

- $|A|$ is the number of members in A .

Cardinality | · |

- $|A|$ is the number of members in A .
- Example: if $A = \{1, 2, 3, 4\}$, then $|A| = 4$.

Cardinality | · |

- $|A|$ is the number of members in A .
- Example: if $A = \{1, 2, 3, 4\}$, then $|A| = 4$.
- If $B = \{1, 2, A\}$, then $|B| = 3$ (because A counts as one member).

Cardinality | · |

- $|A|$ is the number of members in A .
- Example: if $A = \{1, 2, 3, 4\}$, then $|A| = 4$.
- If $B = \{1, 2, A\}$, then $|B| = 3$ (because A counts as one member).

Exercise (set calculations)

Let $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$. Compute:

Exercise (set calculations)

Let $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$. Compute:

① $A \cup B$

Exercise (set calculations)

Let $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$. Compute:

- 1 $A \cup B$
- 2 $A \cap B$

Exercise (set calculations)

Let $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$. Compute:

- 1 $A \cup B$
- 2 $A \cap B$
- 3 $A \setminus B$

Exercise (set calculations)

Let $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$. Compute:

- 1 $A \cup B$
- 2 $A \cap B$
- 3 $A \setminus B$
- 4 $|A|$ and $|B|$

Exercise (set calculations)

Let $A = \{1, 2, 3, 4\}$ and $B = \{2, 3, b, c\}$. Compute:

- 1 $A \cup B$
- 2 $A \cap B$
- 3 $A \setminus B$
- 4 $|A|$ and $|B|$

Ordered pairs and relations

Ordered pairs $\langle a, b \rangle$

- Sets are unordered: $\{a, b\} = \{b, a\}$.

Ordered pairs $\langle a, b \rangle$

- Sets are unordered: $\{a, b\} = \{b, a\}$.
- Ordered pairs are ordered: $\langle a, b \rangle \neq \langle b, a \rangle$.

Ordered pairs $\langle a, b \rangle$

- Sets are unordered: $\{a, b\} = \{b, a\}$.
- Ordered pairs are ordered: $\langle a, b \rangle \neq \langle b, a \rangle$.

Cartesian product $A \times B$

- $A \times B$ is the set of all ordered pairs $\langle x, y \rangle$ with $x \in A$ and $y \in B$.

Cartesian product $A \times B$

- $A \times B$ is the set of all ordered pairs $\langle x, y \rangle$ with $x \in A$ and $y \in B$.
- Example:

Cartesian product $A \times B$

- $A \times B$ is the set of all ordered pairs $\langle x, y \rangle$ with $x \in A$ and $y \in B$.
- Example:

$$\{a, b\} \times \{1, 2\} = \{\langle a, 1 \rangle, \langle a, 2 \rangle, \langle b, 1 \rangle, \langle b, 2 \rangle\}$$

Relations as sets of ordered pairs

- A (binary) relation from A to B is a subset of $A \times B$.

Relations as sets of ordered pairs

- A (binary) relation from A to B is a subset of $A \times B$.
- Example: if a walked b , then:

Relations as sets of ordered pairs

- A (binary) relation from A to B is a subset of $A \times B$.
- Example: if a walked b , then:

$$\langle a, b \rangle \in \llbracket \textit{walked} \rrbracket$$

Relations as sets of ordered pairs

- A (binary) relation from A to B is a subset of $A \times B$.
- Example: if a walked b , then:

$$\langle a, b \rangle \in \llbracket \text{walked} \rrbracket$$

- (This is schematic; we will use more natural relations like *admires*, *visited*, etc. too.)

Relations as sets of ordered pairs

- A (binary) relation from A to B is a subset of $A \times B$.
- Example: if a walked b , then:

$$\langle a, b \rangle \in \llbracket \text{walked} \rrbracket$$

- (This is schematic; we will use more natural relations like *admires*, *visited*, etc. too.)

Truth conditions with relations: *walked*

- Sentence: *Aoife walked Fiona.*

Truth conditions with relations: *walked*

- Sentence: *Aoife walked Fiona*.
- Let $\llbracket \textit{Aoife} \rrbracket = a$ and $\llbracket \textit{Fiona} \rrbracket = f$.

Truth conditions with relations: *walked*

- Sentence: *Aoife walked Fiona.*
- Let $\llbracket \textit{Aoife} \rrbracket = a$ and $\llbracket \textit{Fiona} \rrbracket = f$.
- Translation:

Truth conditions with relations: *walked*

- Sentence: *Aoife walked Fiona.*
- Let $\llbracket \textit{Aoife} \rrbracket = a$ and $\llbracket \textit{Fiona} \rrbracket = f$.
- Translation:

$$\langle a, f \rangle \in \llbracket \textit{walked} \rrbracket$$

Functions

Functions: like relations, but with unique outputs

- A function is a special kind of relation where each input has **exactly one** output.

Functions: like relations, but with unique outputs

- A function is a special kind of relation where each input has **exactly one** output.
- Example idea: *motherOf*(x) returns x 's mother (one individual).

Functions: like relations, but with unique outputs

- A function is a special kind of relation where each input has **exactly one** output.
- Example idea: *motherOf*(x) returns x 's mother (one individual).

Wrap-up

- One-place predicates: sets; predication: membership ($\in$).

Wrap-up

- One-place predicates: sets; predication: membership ($\in$).
- Intersective modification: intersection ($\cap$).

Wrap-up

- One-place predicates: sets; predication: membership ($\in$).
- Intersective modification: intersection ($\cap$).
- Two-place predicates: relations (sets of ordered pairs).

Wrap-up

- One-place predicates: sets; predication: membership ($\in$).
- Intersective modification: intersection ($\cap$).
- Two-place predicates: relations (sets of ordered pairs).
- Functions: special relations with unique outputs.

Wrap-up

- One-place predicates: sets; predication: membership ($\in$).
- Intersective modification: intersection ($\cap$).
- Two-place predicates: relations (sets of ordered pairs).
- Functions: special relations with unique outputs.